SST Codebase Overview

Project 2

ECE/CSC 406/506 Spring 2026

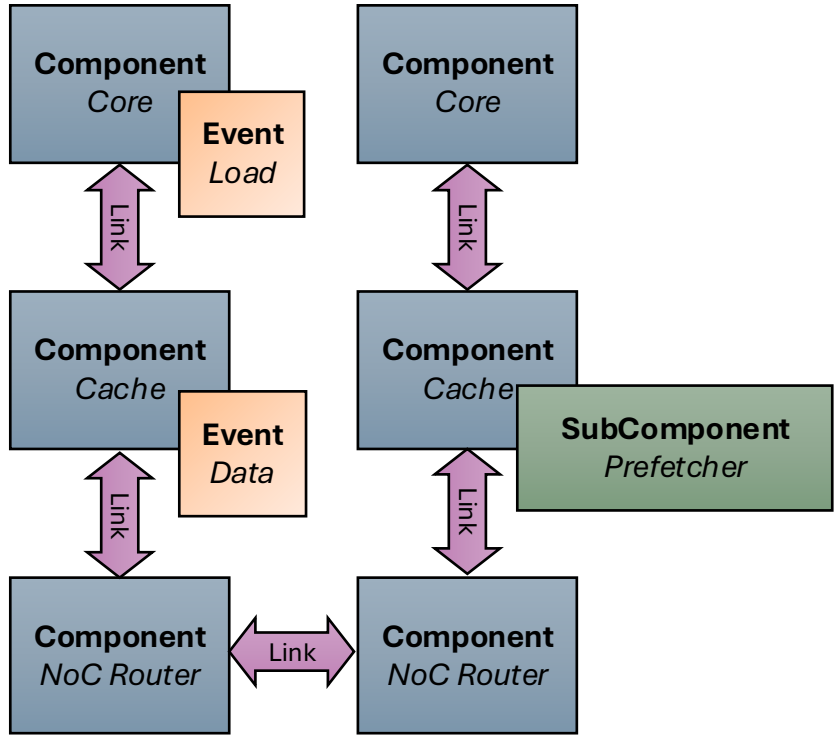
SST BASICS

- SST **Core** framework
  - The backbone of simulation
  - Provides utilities and interfaces for simulation components (models)
    - Clocks, event exchange, statistics and parameter management, parallelism support, etc.
- SST **Element** libraries
  - Libraries of components that perform the actual simulation
  - Elements include processors, memory, network, etc.

Source: ISPASS 2024 presentation on SST

Building Blocks

- SST simulations are comprised of **components**
- Components are connected by **links**
  - Every link has a minimum (non-zero) latency
  - Components define **ports** which are valid connection points for a link
- Components communicate by sending **events** over the links
- Components can use **subcomponents** and **modules** for customizable functionality



Element Library

A collection of components, subcomponents, and/or modules

Source: ISPASS 2024 presentation on SST

$ sst-info memHierarchy | grep " Component"

MemHierarchy: Memory system

- Collection of interoperable memory system elements
  - Caches
  - Directories
  - Memory controllers
    - Interfaces to memory models (DDR, HBM, HMC, NVM, etc.)
  - Scratchpads
  - NoC (network-on-chip) interfaces
  - Buses
- Components are cycle-accurate/cycle-level
- Capable of modeling modern cache and memory subsystems

Source: ISPASS 2024 presentation on SST

MemHierarchy: Cache modeling

$ sst-info memHierarchy.Cache

- Highly configurable
  - Arbitrary hierarchy depth, flexible topologies
  - Cache inclusivity, coherence, private/shared, etc. configurable
  - Single- and multi-socket configurations
  - Prefetch via *Cassini* element library
- Data movement
  - Components support direct, bus, and on-chip network (NoC) communication
- Event types: read/write, atomics, LLSC, noncacheable, custom memory, etc.

Source: ISPASS 2024 presentation on SST

cacheController

- memHierarchy.Cache (cacheController.h / cacheController.cc, built in cacheFactory.cc) is the memory-side component that sits between CPUs and next level toward memory (another cache or memory).
- It owns clock, queues, MSHR, and link managers.
- Acts as a shell that contains CoherenceController subcomponent.
- Handles event dispatching to coherence controller.

CoherenceController

- Shared base class for protocol implementations.
- Contains base event handler functions (handle**) that are overridden by derived classes like MESI_L1.cc.
- Also contains message routing functions:
  - sendOutgoingEvents — Advance local time, dequeue responses and send on down/up links.
  - forwardByAddress— Schedule event toward memory or CPU by address routing.
  - forwardByDestination— Schedule event to its named destination.
  - broadcastMemEventToPeers— Create one MemEvent per peer and queue each by destination. (Hint: Can you use this for read snarfing?)
- Some helper functions:
  - forwardMessage — Clones the triggering MemEvent, sets payload, then calls forwardByAddress. E.g. Called by private L1 to send requests to shared L2.
  - sendResponseUp — Builds response and calls forwardByDestination. E.g. Called by shared L2 to send response to private L1.

Coherence Events

CPU requests	GetS, GetX, GetSX, Write, FlushLine, FlushLineInv, FlushAll	What cores or upper caches inject.
Data responses from lower cache/mem	GetSResp, GetXResp, WriteResp, FlushLineResp, FlushAllResp	Completes outstanding requests.
Replacement / writeback	PutS, PutE, PutM, PutX	Eviction.
Invalidation / intervention	Inv, ForceInv, Fetch, FetchInv, FetchInvX	cache to drop/downgrade/share data.
Responses to Inv/Int	FetchResp, FetchXResp, AckInv	Data or pure ack back to requester.
Flush	ForwardFlush, AckFlush, UnblockFlush	Flush handshake (Not relevant for coherence project)

MESI_L1

- Extends CoherenceController and **implements L1-specific MESI/MSI logic**.
- E.g. handleGetS implementation (Read request):
  - lookup line; if I → processCacheMiss; on success forwardMessage (miss to L2), line → IS, set MSHR.
  - Stable S/E/M: hit path, slice data from line, sendResponseUp to CPU.
- Handles MSHR allocation and also supports atomic LLSC instructions. (Not relevant for coherence project)
- Simple optimizations for part 2 of the project (e.g. change handleGetS behavior for migratory workloads) can be implemented within this file.

I	Invalid — line not valid in this cache
S	Shared — valid, read-only, may exist elsewhere (MESI/MSI).
E	Exclusive clean — valid, writable, no other cache has it.
M	Modified — valid, dirty, this cache is the owner of the data.
IS	Missed read: invalid, GetS sent down, waiting for GetSResp.
IM	Missed write / exclusive: invalid, GetX/GetSX sent, waiting for GetXResp.
SM	Upgrade: line was S, issued upgrade, waiting for GetXResp
***_B**	Line is blocked waiting for a response - flush command related (not part of testcases).

MESI_inclusive

- MESI_Inclusive.cc is the inclusive **shared-cache (L2) implementation**: it maintains sharer/owner metadata, fans out invalidations and downgrades to children, and completes requests with GetSResp/GetXResp.
- Example: handleGetS request from L1
  - I: processCacheMiss, forwardMessage (GetS down to main memory), line → IS.
  - S: Hit, addSharer(req->getSrc), sendResponseUp with line data.
  - E/M:
    - If hasOwner(): must downgrade owner before sharing — downgradeOwner (sends FetchInvX to owner)
    - Else: if no sharers and MESI, grant exclusive with GetXResp and setOwner; else GetSResp and addSharer.
- Helper functions:
  - downgradeOwner: sends FetchInvX to line->getOwner(), waits for ack.
  - invalidateExceptRequestor: Inv to every sharer except the requestor; returns whether any message was sent.
- handleFetchResp / handleFetchXResp / handleAckInv: Complete intervention transactions: merge data, clear owner/sharers, decrement acks, advance M_Inv → M, E_InvX → E, etc.